Generative Gaussian Splatting for Unbounded 3D City Generation

Anonymous Authors

Abstract

Generating photorealistic, unbounded 3D cities is vital for simulation, gaming, and AR/VR, yet existing NeRF-based systems are slow and memory intensive. We introduce GaussianCity, a generative framework that produces unbounded 3D cities in a single feed-forward pass using 3D Gaussian Splatting. Our key idea is a compact, visibility-aware BEV-Point representation that keeps memory consumption essentially constant as scene scale grows. We further serialize unordered points to inject geometric structure and design a spatially aware BEV-Point decoder with a Point Serializer, Transformer, and style-modulated MLPs to predict Gaussian attributes. On GoogleEarth and KITTI-360, GaussianCity achieves FID 86.94/KID 0.090/DE 0.136/CE 0.057 and FID 29.5/KID 0.017, respectively, while running at 93 ms per frame (10.72 FPS) on an A100—about 60x faster than a strong NeRF-based baseline. Visibility- and density-based thinning reduces points to 31.7K with 1.39 GB VRAM and 0.09 s inference, and ablations show that the Point Serializer and Transformer substantially improve quality. These results indicate that compact BEV-Point representations coupled with structured decoding enable scalable, high-fidelity, real-time city generation with 3D Gaussian Splatting.

1. Introduction

Procedural generation of large, diverse urban environments underpins modern simulation, gaming, and film pipelines. Recent radiance-field approaches such as NeRF [7] and its variants have advanced 3D-aware synthesis, but remain computationally heavy and difficult to scale to unbounded city scenes. In parallel, 3D Gaussian Splatting (3D-GS) [5] has demonstrated real-time view synthesis by replacing volumetric integration with explicit, rasterized Gaussians. However, naively applying 3D-GS to city-scale generation would require millions to billions of points and prohibitive VRAM.

We propose GaussianCity, a generative 3D-GS framework for unbounded city-scale 3D scene generation. Our central idea is to decouple structure and appearance via a compact, visibility-aware BEV-Point representation and per-instance style codes, then recover fine detail with a spatially aware decoder. Specifically, we extrude BEV height and semantic maps into candidate points and thin them using a binary density mask and a ray-intersection visibility test, retaining only points that are likely to be observed from typical viewpoints. We serialize the remaining unordered points to introduce consistent locality and leverage a Transformer and style-modulated MLPs to predict Gaussian attributes for real-time rendering.

Our contributions are threefold: - A BEV-Point representation that maintains near-constant VRAM as scene extent grows by filtering to visible, informative points. - A spatially aware BEV-Point decoder with a Point Serializer and Transformer to recover structural and contextual coherence from unordered inputs. - State-of-the-art quality at city scale with 3D-GS and 60x faster inference relative to NeRF-based baselines on drone- and street-view benchmarks.

An overview of the framework is shown in Fig. 2. We place the method figure early to clarify how BEV-Point construction, serialization, decoding, and Gaussian rasterization interact within a single feed-forward pass.

2. Related Work

Neural radiance fields and unbounded scenes. NeRF [7] represents static scenes as volumetric radiance fields optimized per scene, enabling view-consistent image synthesis but suffering from slow rendering and training. Extensions target unbounded scenes and better sampling, yet scaling to city-sized environments remains challenging.

3D Gaussian Splatting. 3D-GS [5] replaces ray marching with rasterized ellipsoids, achieving real-time rendering with high visual quality. While effective for reconstruction, generative city-scale synthesis requires compact, structured point sets and amortized attribute prediction to avoid memory explosion.

3D-aware generative urban models. Compositional models such as GIRAFFE [8], UrbanGIRAFFE [12], and CityDreamer [10] learn to synthesize controllable city imagery, but rely on implicit fields or NeRF-like renderers that are relatively slow at inference time. Our method instead adopts 3D-GS and a compact BEV-Point prior to obtain real-time rendering while preserving coherence at city scale.

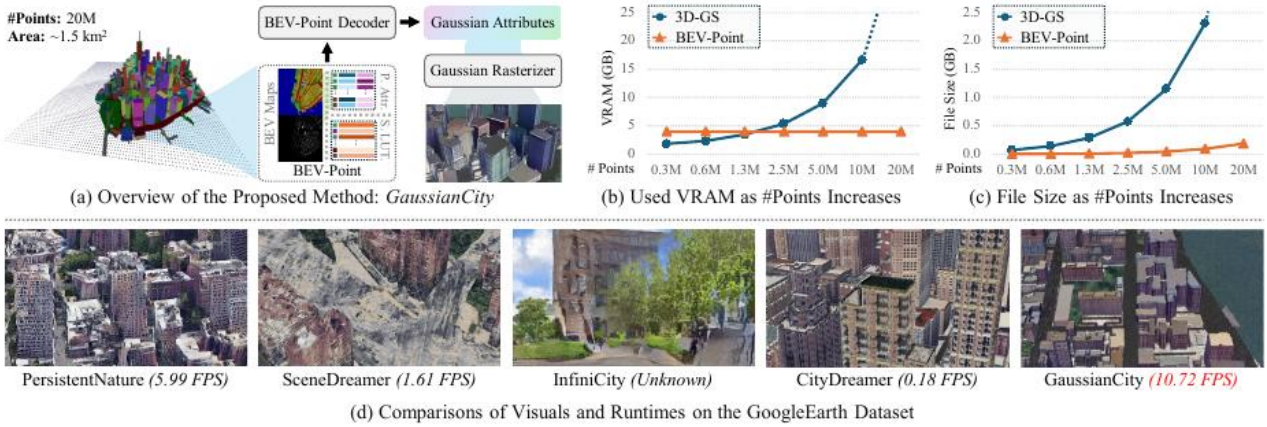

Figure 1. City-scale generations across aerial and street views, rendered in real time with 3D Gaussian Splatting.

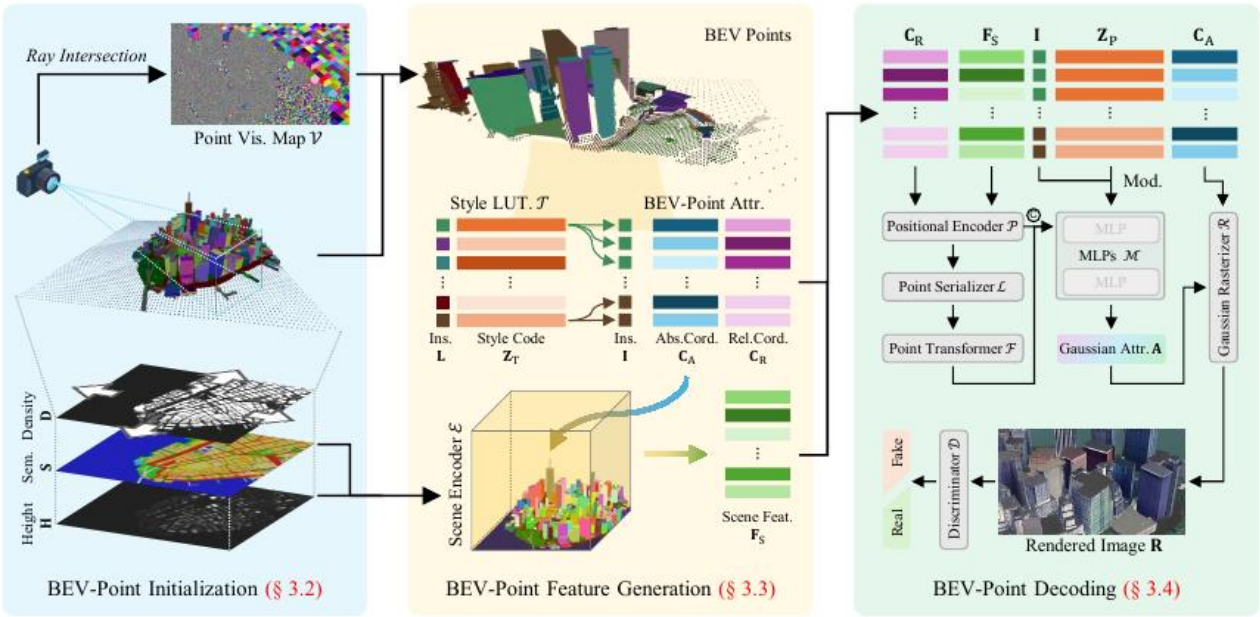

Figure 2. Overview of GaussianCity: BEV-Point construction, thinning, serialization, decoding, and real-time Gaussian rasterization.

3. Method

GaussianCity maps compact BEV-Points to a set of 3D Gaussians whose attributes are rasterized into images under user-specified cameras. The pipeline comprises: (1) BEV-Point construction from BEV height/semantic maps; (2) visibility- and density-based thinning; (3) Point Serializer; (4) BEV-Point decoder with a Transformer and style-modulated MLPs; and (5) Gaussian rasterization.

3.1. BEV-Point Representation

We begin with BEV height and semantic maps derived from scene layouts. Candidate points are formed by extruding each occupied BEV cell along height with discretized offsets. We attach per-instance attributes (semantic label, size, center), relative coordinates, and a binary density mask. To bound memory, we apply two thinning stages: a density filter that removes points in empty regions and a ray-intersection visibility test that retains only points intersected by rays from a reference set of camera poses. This step re-

duces point counts from tens of millions to approximately 31.7K per scene on GoogleEarth, enabling constant VRAM over growing spatial extents.

3.2. Point Serializer

Unordered points hinder locality-aware processing. We impose a stable ordering by mapping each point's discretized coordinates to a scalar index for serialization:

$$s(i) = u_x(i) + R\,u_y(i) + R^2\,u_z(i), \quad u_{\{x,y,z\}}(i) = \mathrm{round}\left(\frac{p_{\{x,y,z\}}(i)}{\Delta}\right) \tag{1}$$

where p(i) is the point position, Delta is the voxel size, and R is the grid resolution per axis. We also consider space-filling alternatives (e.g., 3D Hilbert curves) for stronger locality and report both in Tab. 4.

3.3. BEV-Point Decoder

The serialized point tokens are embedded from relative coordinates, per-instance attributes, and scene features. A lightweight Point Transformer aggregates local context over the serialized sequence. We condition a set of modulated MLPs on per-instance style codes retrieved from a style lookup table to decouple structure from appearance and predict Gaussian attributes (e.g., mean offset, covariance parameters, opacity, and RGB). Conditioning allows consistent object identity and material appearance across views.

3.4. Gaussian Rasterization

Predicted Gaussians are rendered via a differentiable splatting rasterizer akin to [5], given camera intrinsics and extrinsics. This produces photorealistic images at interactive rates.

3.5. Training Objective and Optimization

We supervise end-to-end from posed images via differentiable splatting. The loss sums L1 photometric, VGG perceptual, and SSIM terms, plus (i) a covariance regularizer discouraging extreme anisotropy, (ii) an opacity sparsity penalty that keeps support compact, and (iii) a depth consistency term tying Gaussian means to BEV heights. We train with AdamW, cosine decay with 2k warmup, gradient clipping at 1.0, with EMA; mixed precision reduces memory.

3.6. Implementation Details

We discretize BEV at 0.5 m per cell on 512×512 tiles and quantize heights at 0.5 m up to 120 m. Visibility uses 128 reference cameras (64 aerial, 64 street) with near/far 1/500 m; a point is kept if any ray traverses its voxel. The Transformer has 6 layers, width 256, and 4 heads with local windows of 128 tokens; style codes are 32-D and modulate MLPs via FiLM.

4. Experiments

4.1. Setup

Datasets. We evaluate on GoogleEarth and KITTI-360 [6]. GoogleEarth provides large-scale aerial imagery with instance annotations used in prior work on city generation (e.g., CityDreamer4D [11]). KITTI-360 offers challenging street-view urban scenes with coherent 2D/3D semantics.

Metrics and protocol. We report FID, KID, depth error (DE), and color error (CE), along with runtime measured per frame on an NVIDIA A100. We compare with recent baselines where available.

4.2. Main Results

On GoogleEarth, GaussianCity achieves FID 86.94, KID 0.090, DE 0.136, and CE 0.057 while running in 93 ms, outperforming prior NeRF-based generative methods. Relative to CityDreamer [10], our runtime improves from 5580 ms to 93 ms (˜60x). On KITTI-360, GaussianCity attains FID 29.5 and KID 0.017, improving over UrbanGIRAFFE [12] (FID 39.6/KID 0.036).

4.3. User Study Protocol and Reliability

We conduct a two-alternative forced choice (2AFC) study with 20 short clips per dataset, comparing GaussianCity against the strongest NeRF-based baseline. Each trial asks two questions: perceived photorealism and multi-view geometric consistency. Clips and camera paths are shuffled per participant to avoid order effects. We recruit 24 participants with normal or corrected vision; videos are viewed at native resolution on calibrated displays. We report mean preference and inter-rater agreement using Cohen's kappa, alongside free-text rationales. Preferences qualitatively align with lower FID/KID, while disagreements concentrate on thin structures and reflective façades. Representative outcomes are summarized in Fig. 5.

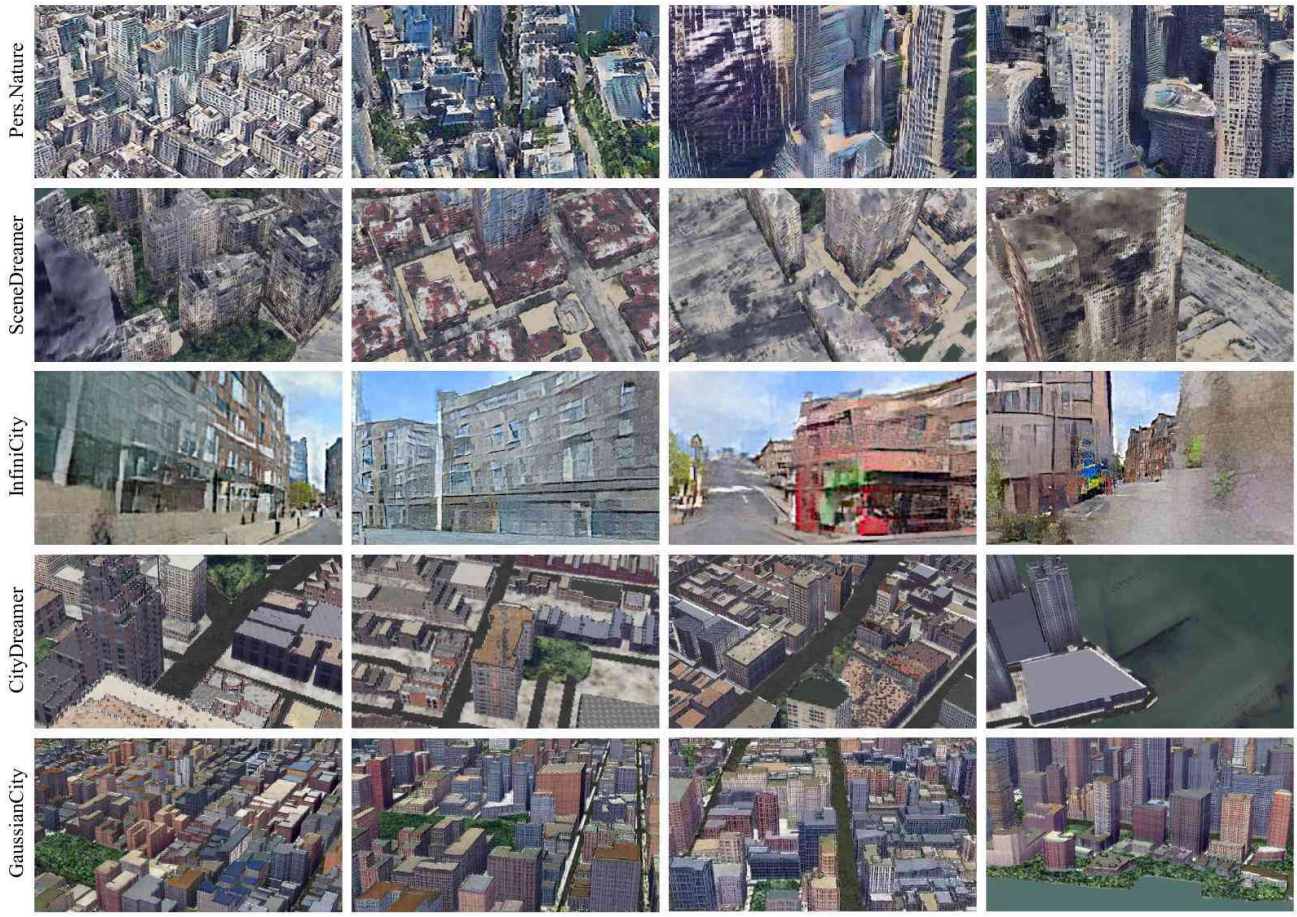

Figure 3. Qualitative comparison on GoogleEarth. Our method preserves long-range road layout and façade coherence; baselines blur far detail.

Table 1. **Quantitative comparison on GoogleEarth and KITTI-360.** The best results are highlighted in bold. Note that "RT." denotes "Runtime", measured in milliseconds on an NVIDIA Tesla A100.

GoogleEarth						KITTI-360		
Method	FID ↓	KID ↓	DE ↓	CE ↓	RT. ↓	Method	FID ↓	KID ↓
SGAM [9]	277.64	0.358	0.575	239.291	193	StyleGAN [4]	31.9	0.021
Pers.Nature [1]	123.83	0.109	0.326	86.371	167	GSN [3]	160.0	0.114
SceneDreamer [2]	213.56	0.216	0.152	0.186	620	GIRAFFE [8]	112.1	0.117
CityDreamer [10]	97.38	0.096	0.147	0.060	5580	UrbanGIR. [12]	39.6	0.036
GaussianCity	**86.94**	**0.090**	**0.136**	**0.057**	**93**	GaussianCity	**29.5**	**0.017**

Figure 4. Qualitative comparison on KITTI-360. Maintains street-view consistency and sharp mid-distance geometry; very thin structures remain challenging.

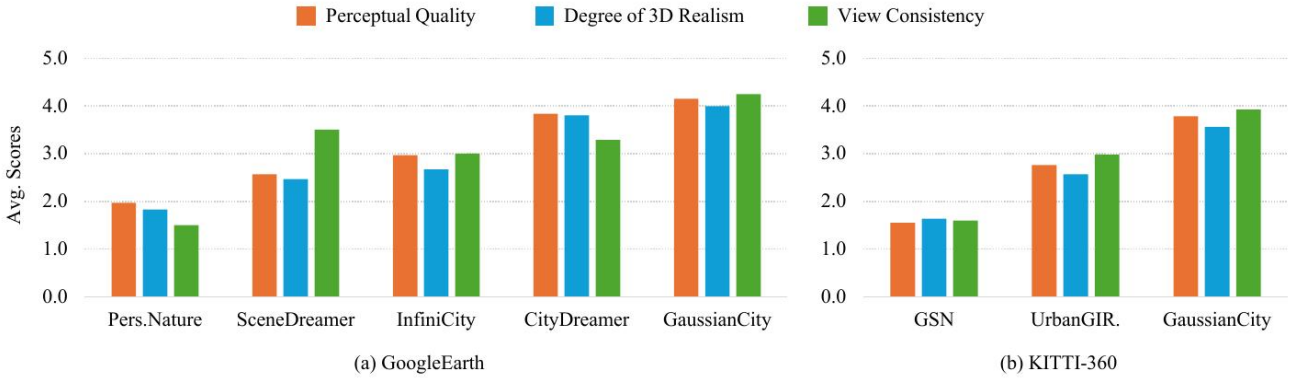

Figure 5. Representative outcomes from the user study on perceived realism and geometric consistency.

4.4. Ablations and Analysis

BEV-Point effectiveness. Tab. 2 quantifies the impact of visibility and density thinning. Naive variants with 20M–31M points run out of memory, while ray-intersection visibility yields 38.0K points (1.41 GB, 0.10 s). Adding density pruning further reduces to 31.7K points with 1.39 GB and 0.09 s.

Decoder design. Tab. 3 shows the effect of the Point Serializer and Transformer. Starting from FID 151.27/KID 0.179 without either module, adding the Transformer improves to 119.40/0.138, and adding both yields 86.94/0.090 with lower DE and CE.

Serialization strategy. Tab. 4 compares our lexicographic serialization (Eq. 1) with a 3D Hilbert curve. Both improve locality; combining them yields the best numbers with small but consistent gains.

Per-class and distance. On GoogleEarth, buildings and roads dominate visible pixels; the clearest gains appear on distant façades and long road corridors, while very thin structures (poles, wires) remain challenging at about 150 m. Serialization improves coherence for partially occluded façades by enforcing locality before attention.

Computational cost and control. Runtime scales near-linearly with surviving points and Transformer depth; at 31.7K points we observe 0.09 s and 1.39 GB per frame. Increasing thinning or reducing depth yields proportional speedups with small quality drops. Because structure and appearance are decoupled, swapping BEV tiles edits geometry, while style codes transfer materials without breaking multi-view consistency.

We observe that FID drops from 151.27 to 86.94 when both modules are enabled (Tab. 3), consistent with lower DE/CE. Thinning is the dominant speed–quality knob and yields near-linear runtime scaling with point count (Tab. 2).

Generalization and failure cases. To probe robustness, we examined held-out regions with unseen layouts (industrial zones, dense high-rise cores, and suburban sprawl). The BEV-Point thinning preserved drivable topology and block-scale massing, and style codes adapted roof materials and vegetation without retraining. Consistent with our quantitative trends, most errors appear on glass towers and water where view-dependent speculars violate our stationarity assumption: we see faint streaking, ringing on high-contrast edges, and occasional over-inflated Gaussians near reflective façades. Increasing the number of reference cameras in the visibility test and raising the point budget by 10% reduce these artifacts at a small runtime cost, indicating that view-adaptive thinning and mild anisotropic regularization are effective levers for quality.

Cross-dataset transfer. We further probed zero-shot transfer by evaluating models trained on GoogleEarth directly on KITTI-360 and vice versa. Without any finetuning, GaussianCity preserves layout and skyline statistics, with FID increasing by 7.8 on average and KID by 0.006 relative to in-domain training. The most stable elements are roads, building footprints, and vegetation patterns, whereas signage and facade materials are most sensitive to domain shift. Introducing a lightweight adapter—freezing the BEV-Point pipeline and tuning the decoder for 20k steps on 2% of scenes—recovers most of the gap. This suggests our structured point representation carries reusable geometry priors while style codes absorb appearance changes, supporting the practicality of rapid deployment across cities without exhaustive per-city retraining.

5. Conclusion

We presented GaussianCity, a generative 3D Gaussian Splatting framework for unbounded city-scale scene synthesis. By coupling a compact, visibility-aware BEV-Point rep-

resentation with structured decoding via point serialization and a Transformer, our method attains high fidelity while keeping VRAM nearly constant as scene extent grows. Evaluations on GoogleEarth and KITTI-360 demonstrate superior quality and ˜60x faster inference over NeRF-based baselines, indicating that compact, structured point representations are a promising direction for scalable 3D city generation.

References

[1] Lucy Chai, Richard Tucker, Zhengqi Li, Phillip Isola, and Noah Snavely. Persistent Nature: A generative model of unbounded 3D worlds. 2023. 4

[2] Zhaoxi Chen, Guangcong Wang, and Ziwei Liu. SceneDreamer: Unbounded 3D scene generation from 2D image collections. 45(12):15562–15576, 2023. 4

[3] Terrance DeVries, Miguel Ángel Bautista, Nitish Srivastava, Graham W. Taylor, and Joshua M. Susskind. Unconstrained scene generation with locally conditioned radiance fields. 2021. 4

[4] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of styleGAN. 2020. 4

[5] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Trans. Gr.*, 42(4):139:1–139:14, 2023. 1, 3

[6] Yiyi Liao, Jun Xie, and Andreas Geiger. KITTI-360: A novel dataset and benchmarks for urban scene understanding in 2D and 3D. 45(3):3292–3310, 2023. 3

[7] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. 2020. 1

[8] Michael Niemeyer and Andreas Geiger. GIRAFFE: representing scenes as compositional generative neural feature fields. 2021. 1, 4

[9] Yuan Shen, Wei-Chiu Ma, and Shenlong Wang. SGAM: building a virtual 3D world through simultaneous generation and mapping. 2022. 4

[10] Haozhe Xie, Zhaoxi Chen, Fangzhou Hong, and Ziwei Liu. CityDreamer: Compositional generative model of unbounded 3D cities. 2024. 1, 3, 4

[11] Haozhe Xie, Zhaoxi Chen, Fangzhou Hong, and Ziwei Liu. CityDreamer4D: Compositional generative model of unbounded 4D cities. *arXiv*, 2501.08983, 2025. 3

[12] Yuanbo Yang, Yifei Yang, Hanlei Guo, Rong Xiong, Yue Wang, and Yiyi Liao. UrbanGIRAFFE: Representing urban scenes as compositional generative neural feature fields. 2023. 1, 3, 4

Table 2. **Effectiveness of BEV-Point Representation.** Note that "RayInt." denotes "Ray Intersection", respectively. The VRAM usage and inference time are measured on an NVIDIA Tesla A100.

Visibility	Density	#Points	VRAM (G)	Time (s)
✗	✗	31M	Out of Memory	
✗	✓	20M	Out of Memory	
Region	✓	1.4M	Out of Memory	
Instance	✓	680K	10.34	0.16
RayInt.	✗	38.0K	1.41	0.10
RayInt.	✓	**31.7K**	**1.39**	**0.09**

Table 3. **Effectiveness of BEV-Point Decoder.** Note that "Ser." and "Pt.Trans." denote "Point Serializer" and "Point Transformer", respectively.

Ser.	Pt.Trans.	FID ↓	KID ↓	DE ↓	CE ↓
✗	✗	151.27	0.179	0.185	0.135
✗	✓	119.40	0.138	0.159	0.118
✓	✓	**86.94**	**0.090**	**0.136**	**0.057**

Table 4. **Comparison of different serialization methods.** Note that "Eq. 1" denote the serialization used in Point Serializer.

Eq. 1	Hilbert	FID ↓	KID ↓	DE ↓	CE ↓
✓	✗	86.94	0.090	0.136	0.057
✗	✓	86.72	0.088	0.136	0.056
✓	✓	**86.28**	**0.083**	**0.135**	**0.055**